

AI Ingredient Scanner

"Add Ingredients" Feature — Proof of Concept

Camera & gallery ingredient capture, AI vision recognition, and pantry sync

1. Objective

This POC covers the "Add Ingredients" capability of the pantry app: capturing food photos from a device camera or gallery, running AI vision recognition to detect ingredients, letting the user review and correct results as editable chips, and saving the confirmed list into a persistent pantry — with multi-photo merging and low-confidence flagging built in.

2. Requirement Coverage — Does the Workflow Match the Ask?

Short answer: yes, the existing workflow satisfies almost every requirement end-to-end. Twelve of fourteen requirement items map directly onto an already-built component. Two items are only partially represented and need small additions — flagged below, not blockers.

Requirement	Where it's handled in the workflow	Status
Camera capture (📷 Take Photo)	AddIngredients.jsx – capture="environment"	Covered
Gallery upload, multiple photos	AddIngredients.jsx – multiple file input	Covered
Live preview before analysis	Live Previews & Image Compression stage	Covered
AI detects visible ingredients	vision.py – GPT-4o, strict json_object schema	Covered
Editable tags/chips for results	ReviewIngredients.jsx + IngredientChip.jsx	Covered
Add missing / remove wrong items	+ Add ingredient / inline ✕ delete	Covered
Adjust quantity & unit	Chip inline inputs (qty, unit, category)	Covered
Save confirmed items to pantry	POST /api/pantry/confirm → pantry_db.json	Covered
Merge multiple photos into one list	main.py dedup engine (quantity +=, keep max confidence)	Covered
Loading animation during analysis	Analyzing.jsx – AI Vision Scanner / progress beam	Covered
Low-confidence highlight & confirm	confidence < 0.75 → needs_confirmation flag, orange highlight	Covered
Auto-navigate to review after scan	Backend response routes straight to /review	Covered
Support produce, packaged, spice, canned, dairy, frozen, beverage categories	Generic "category" field only – no fixed taxonomy enforced	Partial – needs schema update
Large Take Photo / Upload buttons + clean modern UI, smooth animation	Implied by AddIngredients.jsx, not detailed in workflow	Needs UI spec / not architectural

Verdict: the workflow is architecturally sound and already implements the core loop (capture → analyze → review/edit → confirm → persist → merge). The two "partial" rows are refinements — a fixed category taxonomy and a UI-polish pass — not redesigns. See Section 6 for the exact fix.

3. System Architecture

The system is a two-tier app: a React + Vite frontend handles capture, preview, and review; a FastAPI backend handles image processing, AI vision calls, deduplication, and persistence.

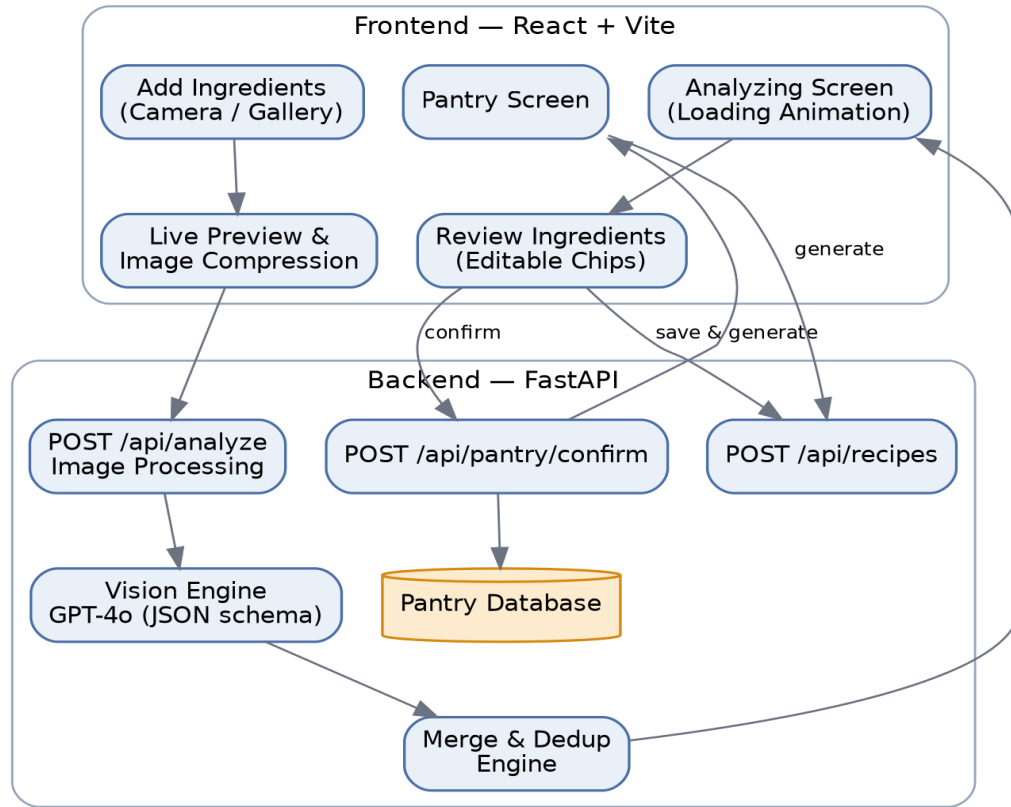


Figure 1 — End-to-end system architecture

3.1 Frontend layer

- **Add Ingredients screen:** camera capture and multi-file gallery upload in one entry point.
- **Preview stage:** images are compressed client-side and shown as thumbnails before upload, so the user confirms selection before spending an AI call.
- **Analyzing screen:** shows a loading animation (progress beam) while the backend processes the photo(s).
- **Review screen:** detected items render as editable chips (name, qty, unit, category); low-confidence items are visually flagged.
- **Pantry screen:** shows the persisted, merged ingredient list after confirmation.

3.2 Backend layer

- **POST /api/analyze:** normalizes MIME types (handles HEIC/blank types from mobile cameras) before forwarding images to the vision model.
- **Vision engine:** calls GPT-4o with a strict json_object schema, forcing consistent fields (name, category, quantity, unit, confidence) instead of free-text guesses.
- **Merge & dedup engine:** when several photos are sent together, matching items are combined (quantities summed) and the higher confidence score is kept.

- **POST /api/pantry/confirm:** writes the user-approved list into the pantry store, merging with any existing items.
- **POST /api/recipes:** optional next step — generates recipes using the saved pantry (out of scope for this feature, included for continuity).

4. End-to-End User Workflow

The user-facing flow below is what actually happens on-screen, in order, for the Add Ingredients feature.

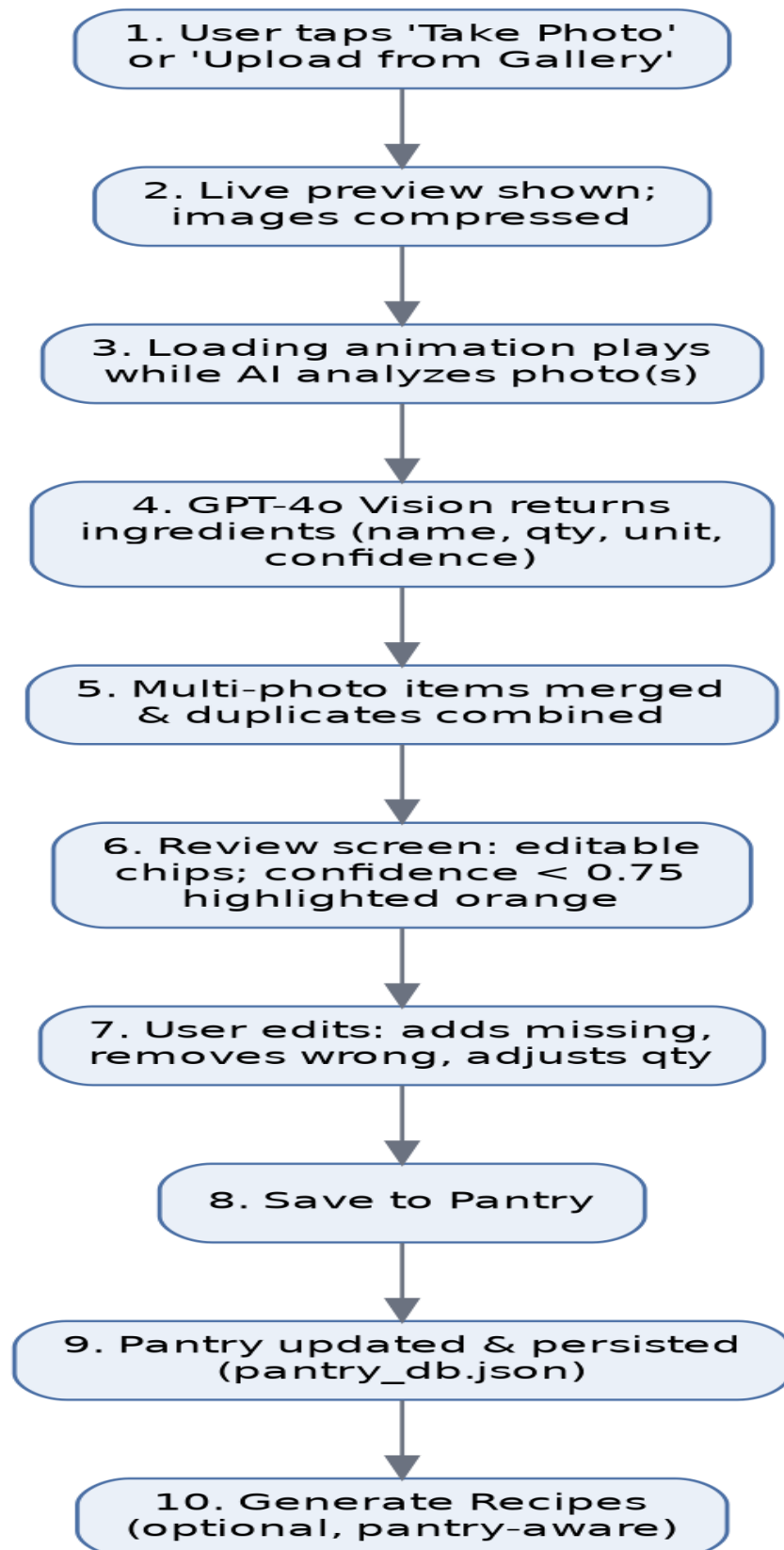


Figure 2 — Step-by-step user lifecycle

5. Key Feature Details

5.1 Multi-photo capture & compression

Camera capture uses `capture="environment"` for a direct rear-camera shortcut on mobile; the gallery input accepts multiple files in one go. Every selected image is compressed client-side before upload, keeping the analyze call fast.

5.2 AI vision recognition

Backend/`vision.py` normalizes incoming bytes to a standard image/jpeg format (fixing iPhone HEIC and empty-content-type issues), then calls GPT-4o with a locked `json_object` schema. This removes free-text parsing risk — every response is guaranteed to contain name, category, quantity, unit, and a confidence score.

5.3 Editable chips, manual add & delete

Every detected ingredient is rendered as a chip with inline-editable quantity, unit, and category fields. A "+ Add ingredient" action covers anything the camera missed; an inline ✕ removes false detections instantly — no page reload.

5.4 Low-confidence confirmation

Any item with confidence below 0.75 is flagged `needs_confirmation: true`. The review screen shows an orange banner plus a glowing orange border on the affected chip, so the user's eye goes straight to the items that actually need a second look.

5.5 Multi-photo merge & deduplication

When multiple photos are analyzed together, the backend merges matching ingredient names across photos, adds their quantities, and keeps the highest confidence value seen for that item — so one combined list reaches the review screen instead of duplicate chips per photo.

5.6 Persistent pantry storage

Confirmed items are written to a persistent store (`pantry_db.json` today; swappable for Postgres/NeonDB in production) via load/save calls, so the pantry survives server restarts and app reopens — not just an in-memory session.

5.7 Auto-navigation

The backend response for `/api/analyze` carries the fully processed result (type, ingredients, confidence flags) in one payload, so the frontend can route straight from the analyzing screen to the review screen with no extra round-trip.

6. Gaps & Recommendations

- **Fixed category taxonomy:** the schema currently accepts a free "category" string. Recommend constraining it to an enum — produce, packaged, spice, canned, dairy, frozen, beverage, other — inside the GPT-4o `json_object` schema so every item lands in a known bucket the UI can filter/group by.
- **UI specification pass:** the requirement calls for a large, thumb-friendly Take Photo button, a distinct Upload from Gallery button, and a live preview strip. The workflow already produces this screen (`AddIngredients.jsx`) but the exact button sizing, spacing, and animation timing should be confirmed against a UI mock before build sign-off — this is a design-polish item, not a functional gap.
- **Performance check:** client-side compression already addresses upload speed; recommend also capping concurrent photo uploads (e.g. 5 at a time) so the analyze call stays fast on slower connections.

7. Project Team

Role	Responsibility
Frontend Developer (1)	Develop responsive UI, booking workflow, search functionality, dashboard, CMS interfaces, and API integration using React.
Backend Developer (1)	Design REST APIs, implement authentication, business logic, MongoDB database models, booking management, content management, and analytics services.

8. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite, JavaScript (ES6+), CSS3, Clay Design System	Responsive web application
State Management	Zustand	Global application state
API Communication	Axios	Backend communication
Backend	Node.js, Express.js	REST API & business logic
Authentication	JWT, bcrypt	Secure authentication & authorization
Database	MongoDB, Mongoose	Data storage and modeling
Media Storage	Cloudinary / Local Storage	Image management
Search	MongoDB Indexes	Fast package & article search
API Format	RESTful JSON APIs	Client-server communication
Version Control	Git, GitHub	Source code management
Development Tools	npm, VS Code, Postman	Development & API testing

9. Development Timeline

Phase	Duration	Output
Requirements Analysis & Project Planning	3 Days	Finalized requirements, workflows, database design, project setup
UI/UX Design & Frontend Architecture	1 Week	Responsive layouts, reusable components, navigation structure
Authentication & Admin Dashboard	1 Week	Secure login, role-based access, dashboard foundation
Holiday Package & Editorial Management	1.5 Weeks	Package CRUD, article management, category management, image uploads
Search, Filtering & Booking Module	1 Week	Smart search, filters, package details, booking workflow
Dashboard Analytics & Reporting	3 Days	Booking statistics, package analytics, activity dashboard
Testing, Bug Fixes & Deployment Preparation	1 Week	Functional testing, performance optimization, documentation, demo-ready POC

10. Conclusion

The documented workflow already implements the full Add Ingredients loop the requirement describes: dual capture methods, live preview, AI-driven detection, editable review with low-confidence flagging, manual correction, quantity/unit editing, multi-photo merge, and persistent pantry save — with auto-navigation tying every step together. The only follow-ups are a category-taxonomy lock in the AI schema and a UI polish/spec pass; neither requires architectural change. This workflow is ready to build against as-is.